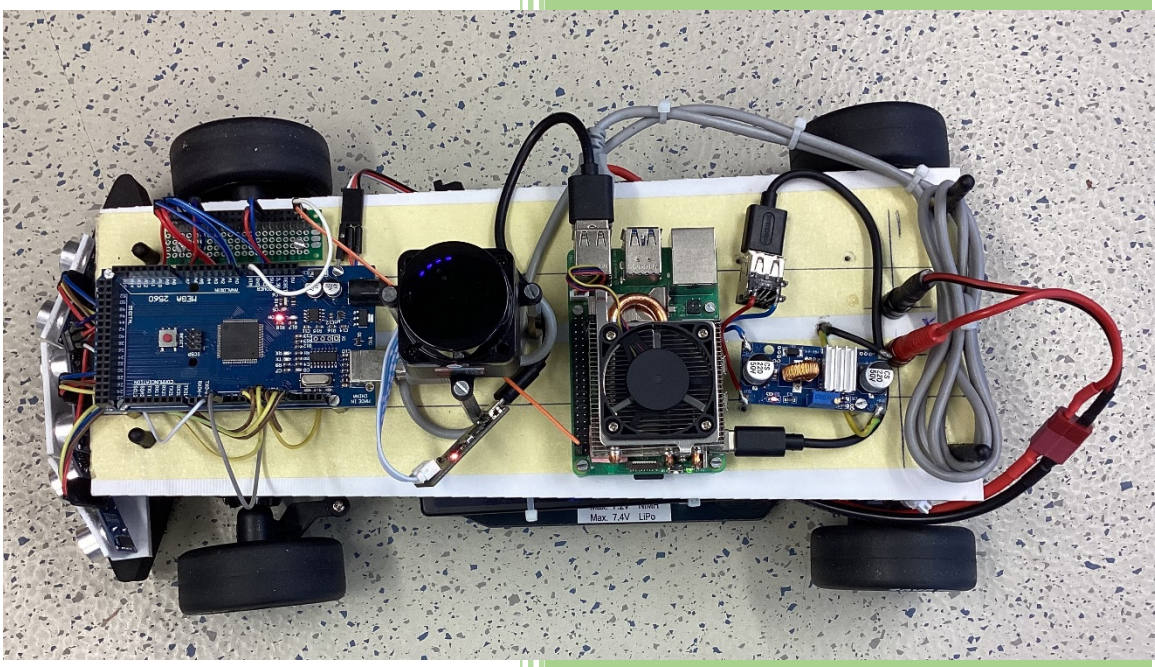
Autonomes RC Auto



Shad Mir, Yannik, Manuel, Jenny

TGI-2

23.06.2025

Inhalt

Aufgabenbeschreibung 1
Technologieschema 1
Schaltbild 2
Projektbeschreibung 2
Hardware 2
Bodykit: 2
Reifen: 3
Motor: 3
Servo/ Lenkung: 3
LiDAR: 3
Raspberry Pi 5: 3
Mikrokontroller: 4
Ultraschallsensoren: 4
Batterie: 4
Zusammenarbeit der Komponenten: 4
Software 5
ROS2: 5
Nav2: 5
Duplex-Kommunikation: 5
Arduino: 5
Projektablauf 6
Einige Problematiken: 7
Programmcode 8
Ressourcen 9
Datenblatt Ultraschallsensoren 9
Quellen 11

Aufgabenbeschreibung

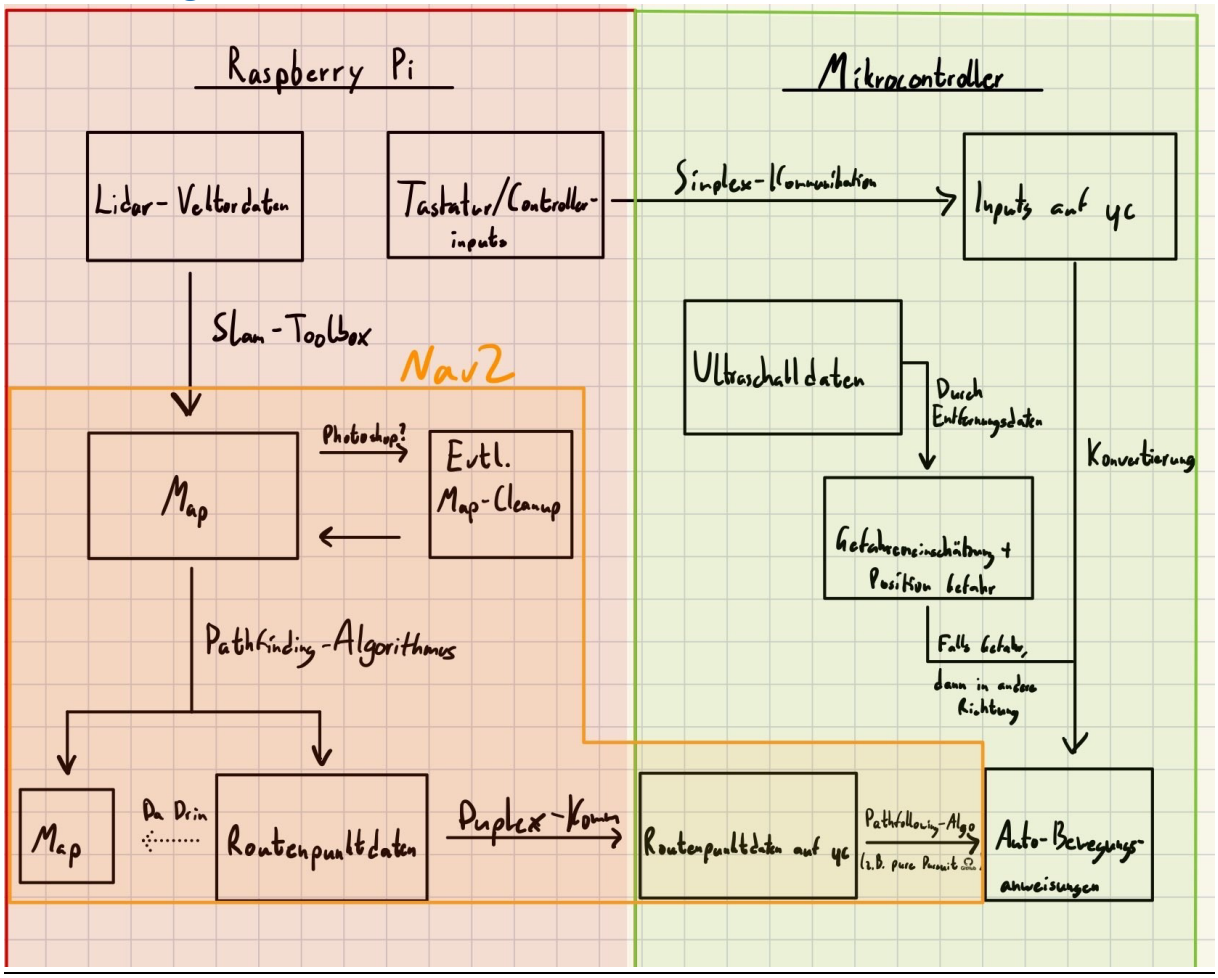
Anforderungen an die Hard- und Software:

- Verwendung von mind. zwei Interrupts

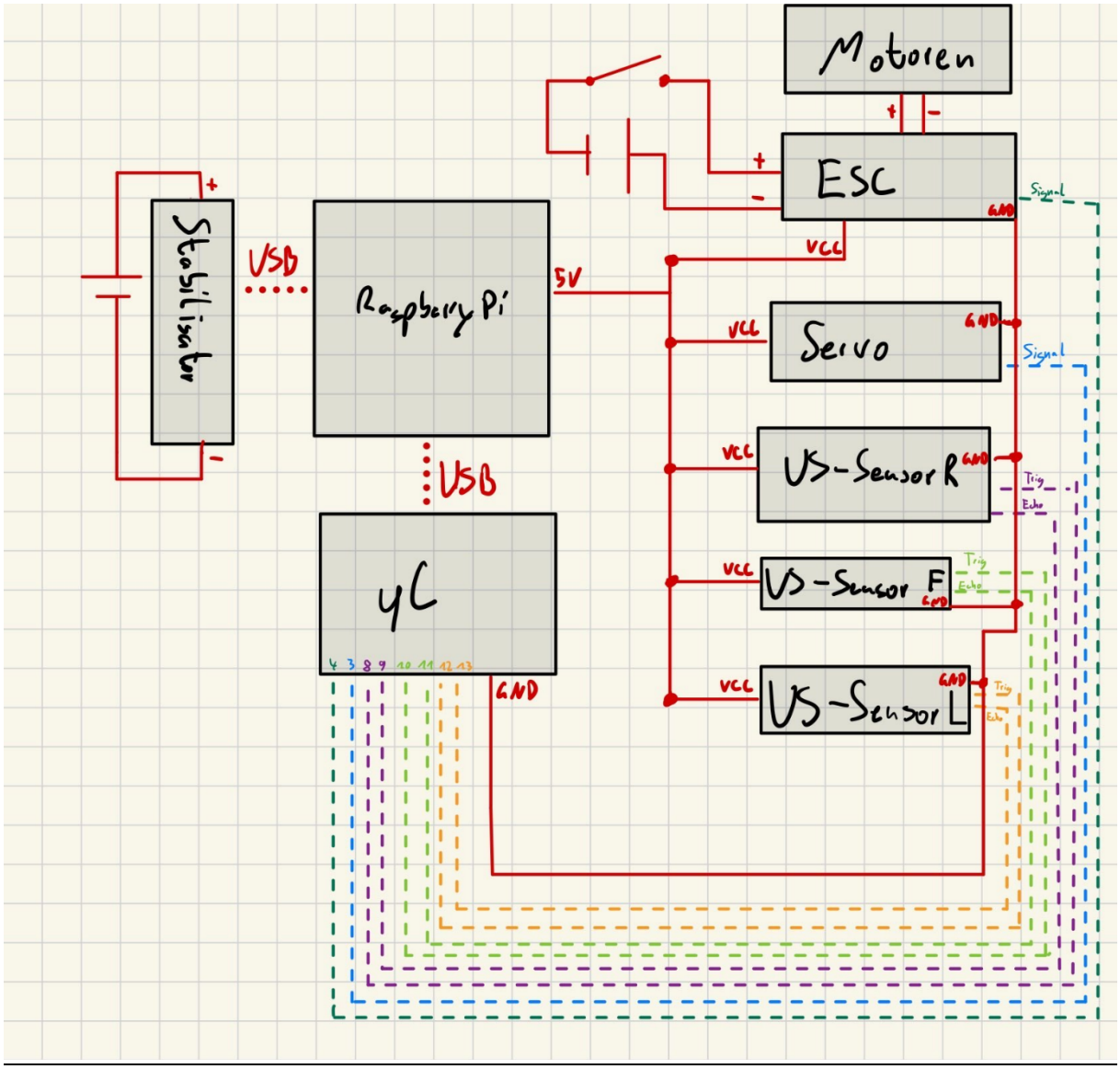
  Oder

- Verwendung von mind. ein Interrupt und mind. eine Datentabelle
- Kern des Projekts ist die Software, also das erstellte Programm. Am Ende des Projekts sollte jedes Gruppenmitglied in der Lage sein die Algorithmen und die Funktionsweise des Projekts zu erklären.

Technologieschema



Schaltbild



Projektbeschreibung

Unser Ziel war es, dass sich das Auto autonom an einen gezielten Ort bewegt. Dafür haben wir vier benötigte Komponenten an das Auto angebaut. Das Ergebnis ist eine live selbsterstellende Karte der Umgebung, ein Frühwarnsystem gegen unvorhersehbare Hindernisse und schlussendlich ein sich autonom fortbewegendes Modell-Auto.

Hardware

Bodykit:

Das FS53113CEI-Chassis ist ein grundlegender Baustein eines RC-Autos. Das Chassis beinhaltet die Bodenplatte, Achsen und evtl. Gearbox, Stoßdämpfer und Halterungen sowie andere Bauteile zur Stabilisation und Befestigung verschiedenster Elemente. Wir verwenden dieses Chassis aufgrund von dem AWD-Setup, der einfachen Möglichkeit zum Anbau eigener Teile und der großen Menge an Stoßdämpfer und Schutz des Chassis gegen Aufprall.

Reifen:

Wir verwenden Harilla RC Rally Reifen im Maßstab 1:10 mit einer 12 mm Nabe. Die Reifen bestehen aus hochverschleißfestem Gummi und wurden ursprünglich für Tourenwagen wie den XV01 konzipiert.

Durch ihre Drift-taugliche Bauweise bieten sie einen guten Kompromiss zwischen Bodenhaftung und Wendigkeit, was sowohl für präzises Navigieren als auch für Richtungswechsel im autonomen Betrieb vorteilhaft ist. Die Standardgröße sorgt für Kompatibilität mit unserem Chassis und erlaubt einfache Montage.

Motor:

Der Fasizi ELA13305 Motor mit wasserdichtem ESC 60A ist ein Brushless-Motor speziell für RC-Autos entwickelt. Obwohl die Bedienung eines Brushless-Motors im Gegensatz zu einem Brushed-Motor deutlich komplizierter ist, gleichen die Vorteile in der Verwendung bei unserem Auto dies aus. Der Motor ist deutlich effizienter, hat eine längere Lebensdauer mit höherer Drehzahl und leitet Wärme besser ab. Deshalb findet der Brushless-Motor bei uns im Auto seine Verwendung als einziger Motor in unserem AWD- (All-Wheel-Drive) -Chassis.

Servo/ Lenkung:

Der Miuzei 20KG Servo Motor wird in unserem Projekt zur Steuerung der Lenkung verwendet. Ein Servo kann präzise in eine bestimmte Position gebracht werden, was für die Lenkbewegung ideal ist.
Im Auto übernimmt der Servo also die Funktion des Lenksystems, durch das der Kurs angepasst wird z. B. bei einem Hindernis oder einer Wegänderung.

LiDAR:

Ein LiDAR ist eine rotierende Komponente, der Lichtsignale aussendet, die Zeit bis zur Rückkehr der Lichtstrahlen misst und daraus Punkte im Raum feststellt, die Wände oder andere Objekte sind.

Wir verwenden einen Waveshare D500 Developer Kit.

Die Aufgabe in unserem Auto besteht darin uns Informationen der Umgebung zu liefern und dem Raspberry Pi alle nötigen Daten zum Aufenthaltsort des Autos zu besorgen.

Raspberry Pi 5:

Der Raspberry Pi 5 ist die neueste Generation des Raspberry Pis. Dieser ist ein Einplatinencomputer, der alleinständig als Computer funktionsfähig ist. Er kann komplexe Berechnungen dank seines leistungsstarken Prozessors und 8 GB RAM mühelos ausführen. Mithilfe von Wlan kann man auf ihn auch ohne Physikalische Verbindung von einem anderen Gerät aus zugreifen.

Die Aufgabe in unserem Auto besteht darin, alle nötigen Kalkulationen für die Wegberechnung auszuführen, als Schnittstelle des Programms für die Interaktion mit dem Benutzer zu dienen und als Graphische Einheit für die Darstellung der Karte und ähnlichem zu dienen.

Mikrokontroller:

Der Mikrokontroller Arduino Mega 2560 ist einer der größeren Mikrokontroller von Arduino. Er verfügt über 54 digitale IO Pins von denen 15 für PWM verwendet werden können, 16 analoge Input Pins, 4 UART-Pins, USB-Anschluss und einen 16 MHz Kristall-Oszillator.

Er findet bei uns Verwendung aufgrund seiner Vielzahl an Pins (vor allem PWM-Pins) und dem USB-Anschluss für einfachere Verbindung mit dem Raspberry Pi.

Der Aufgabenberiech des Mikrokontrollers in unserem Auto lässt sich in zwei Teile unterteilen.

Zum einen die Schnittstelle zwischen den Ultraschallsensoren und dem Auto (Mikrokontroller kann Auto bei Frühwarnung schnell umlenken) und zum anderen ist der Mikrokontroller die Verbindung zwischen Bewegungsanweisungen des Raspberry Pis und den Motoren.

Der Mikrokontroller steuert also die Motoren an, nachdem er, per Duplex-Kommunikation, Anweisungen vom Raspberry Pi erhält.

Ultraschallsensoren:

Die Ultraschallsensoren vom Modell HC- SR04 sind gängige Sensoren für die Verwendung in Kombination mit Mikrokontrollern.

Sie geben eine Schallwelle aus, die dann wieder eingefangen wird. Die Zeit dazwischen wird gemessen und verwendet, um den Abstand eines Hindernisses herauszufinden.

Die Sensoren können ungefähr 15° Bereiche vor sich auf eine Entfernung von zwei bis vier Metern erkennen.

Dies können wir nutzen, indem wir drei Sensoren nebeneinander an der Front des Autos anbringen und somit einen ca. 45° Winkel vor dem Auto abdecken. Die Aufgabe der Sensoren in unserem Auto ist also das Frühwarnungssystem gegen plötzlichen Hindernissen, die Unvorhergesehen sind, wie zum Beispiel Personen am Vorbeilaufen.

Ein Datenblatt kann im Ressourcenteil unter Datenblatt Ultraschallsensoren gefunden werden.

Batterie:

Für die Stromversorgung unseres RC-Autos verwenden wir einen Zeee 2S LiPo-Akku mit 5200 mAh, 7,4 V und 100 C Entladerate. Der Akku liefert zuverlässig Energie für den Brushless-Motor, die Servo-Lenkung und den Raspberry Pi, der über einen Step-Down-Wandler mit 5 V versorgt wird.

Dank der hohen Entladerate von 100 C ist auch bei plötzlichem Leistungsbedarf – etwa beim Beschleunigen – eine stabile Stromversorgung gewährleistet. Das robuste Hardcase schützt den Akku vor Beschädigungen im Fahrbetrieb.

Der Anschluss erfolgt über einen T-Plug (Deans-Stecker), der bei Bedarf mithilfe eines Adapters an XT60-kompatible Komponenten angepasst wurde.

Zusammenarbeit der Komponenten:

Alle Komponenten im Fahrzeug arbeiten eng zusammen, um autonomes Fahren zu ermöglichen. Der Raspberry Pi übernimmt die Hauptverarbeitung und Berechnung, basierend

auf den Daten, die er vom LiDAR, den Ultraschallsensoren und dem Mikrokontroller erhält. Der Mikrokontroller setzt diese Befehle in konkrete Motorsteuerungen um und wirkt gleichzeitig als schnelles Frühwarnsystem. Sensoren erkennen die Umgebung, die Batterie versorgt alles mit Strom, und das Bodykit hält alles zusammen.
Die Zusammenarbeit stellt sicher, dass alle Teile effizient kommunizieren und auf Veränderungen in der Umgebung reagieren können.

Software

ROS2:

ROS2 (Robot Operating System 2) ist die Middleware unseres Projekts und stellt alle grundlegenden Funktionen für die Kommunikation zwischen den einzelnen Software-Komponenten bereit. Die Publisher/Subscriber-Architektur erlaubt es dem Raspberry Pi, Sensordaten zu empfangen, Entscheidungen zu treffen und diese an den Mikrokontroller weiterzuleiten. Besonders vorteilhaft ist die Unterstützung von Echtzeitkommunikation und das Nav2-Stack-Modul.

Nav2:

Nav2 ist ein ROS2-Paket zur Navigation autonomer Roboter. Es erlaubt es uns, mittels SLAM die Umgebung zu kartieren und Pfade zu planen. Die Lokalisierung erfolgt mithilfe des LiDAR-Sensors, während das Pathfinding dynamisch auf Hindernisse reagieren kann. Die Integration mit dem LiDAR-Modul und den Ultraschallsensoren sorgt für ein robustes Navigationsverhalten in Echtzeit.

Duplex-Kommunikation:

Unter Duplex-Kommunikation versteht man den beidseitigen Datenaustausch zwischen zwei Geräten. In unserem Fall kommunizieren der Raspberry Pi und der Mikrokontroller über eine serielle Schnittstelle (z. B. USB oder UART) miteinander.
Die Duplex-Kommunikation sorgt dafür, dass der Raspberry Pi dem Mikrokontroller Bewegungsanweisungen geben kann und dieser wiederum Rückmeldungen wie Sensorwerte oder Statusinformationen zurückschickt.

Arduino:

Der Arduino Mega 2560 wurde mit der IDE programmiert, um sowohl die Ultraschallsensoren auszuwerten als auch über PWM-Signale den Motor und den Servo zu steuern. Die Befehle vom Raspberry Pi werden über eine serielle Schnittstelle empfangen und interpretiert. Zusätzlich wurde ein Interrupt-basiertes System zur schnellen Reaktion bei Hinderniserkennung implementiert.

Projektablauf

Unsere Fortschritte bis zum Start des Projekts (ohne genaue Zeitpunkte):

- Einarbeitung in Programmiersprachen und Libraries
- Konstruktion des Grundaufbaus des Fahrzeugs
- Modifikation und Anpassung der Fahrzeugkomponenten
- Entwurf des finalen Fahrzeugaufbaus
- Recherche zu Bauteilen, Libraries und interner Kommunikation
- Kompatibilitätsprüfung von Hard- und Software
- LiDAR-Tests zur Abstandsmessung
- Entwicklung einer einheitlichen Softwareschnittstelle
- Tests zur Steuerung über verschiedene Eingabegeräte
- Entwicklung und Test eines Kurzzeit-Warnsystems (NUCLEO 64 + Ultraschall)
- Vorbereitung und Konfiguration des Raspberry Pi
- Erprobung unterschiedlicher Pathfinding-Algorithmen
- Fallspezifische Optimierung der Algorithmen

Terminplan nach Start des Projekts:

Aufgabe	**Termin**
Blockschaltbild	17.01.2025
Datenflussschema	30.01.2025
Auto Layout	15.03.2025
Auto Befestigung	16.03.2025
Stromversorgung von Mikrocontroller und Raspberry Pi	19.03.2025
Platine löten	19.03.2025
Technologieschaltbild	20.03.2025
Kommunikation mit Mikrocontroller	20.03.2025
Auto Halterung	20.03.2025
Kommunikation Mikrocontroller und Raspberry Pi	20.03.2025
Terminplan	20.03.2025
Code Ultraschallsensoren	21.03.2025
Projektstatus 1	10.04.2025
SLAM Mapping	01.05.2025
Projektstatus 2	15.05.2025

Einige Problematiken:

Dieser Abschnitt soll der Beschreibung einiger Probleme dienen, auf die wir gestoßen sind und wie wir sie gelöst haben.

Problem 1: Falsche Versionen oder keine Kompatibilität:

Da wir für unser Auto mehrere Bibliotheken verwenden, ist es natürlich kein Erstaunen und auch in unserer Erwartung, dass es Anfangs Kompatibilitätsprobleme gibt. Mit Problemen in diesem Ausmaß konnte aber keiner rechnen. Viele Libraries, die wir verwenden, haben fehlende Integration zur Raspberry Pi 5 Architektur. Ein Beispiel dafür sind die Libraries für den LiDAR. Dies wurde behoben, indem Shad Mir den Library-internen Code verändert hat, um die Library kompatibel zu machen. Im Beispiel der LiDAR-Library wurden Shad Mirs Veränderungen 2 Wochen danach auch offiziell genau so übernommen, um den Support vom Pi 5 zu gewährleisten. Auch sonst wurden stark unkonventionelle aber nützliche Wege gewählt, um die Kompatibilität mit der richtigen Version zu gewährleisten. Zum Beispiel nimmt eine Library, die wir verwenden, an, dass sie auf einem anderen Gerät installiert ist als auf einem Raspberry Pi 5. Zusammenfassend haben wir also inkompatible Bibliotheken durch Manipulation der Bibliothek kompatibel gemacht.

Problem 2: Einbruch der Stromversorgung bei interner Stromversorgung:

Damit unser Auto sich autonom fortbewegen kann, muss natürlich auch die Stromversorgung intern verbaut sein. Wir können also nicht auf den stabilen Strom des Kabels aus der Steckdose angewiesen sein. Unsere erste Lösung waren einfach zwei LiPo-Akkus, einen für den Motor und einen für den Raspberry-Pi und den Rest, die direkt angeschlossen wurden. Die Problematik ist hier direkt aufgetreten. Spannungs- und Stromstärkenschwankungen haben dem Raspberry Pi nicht ausgereicht und dieser hat sich infolgedessen öfters ausgeschaltet. Unser zweiter Ansatz war eine Powerbank statt der Akkus. Die Problematik hierbei ist aber, dass unser Stromverbrauch zu hoch war. Die Stromstärke der Powerbank hat nicht ausgereicht, um unseren kompletten Stromkreis angemessen zu versorgen. Unser Lösungsansatz hier war der Bau einer 5A-Powerbank. Da dies aber zu viel Aufwand ist, haben wir uns für einen ähnlichen Ansatz entschieden. Wir haben einen Stabilisator an unsere LiPo-Akkus angebaut, der Schwankungen in Spannung und Stromstärke verringert hat. Somit haben wir die Stabilität einer Powerbank mit der Stromstärke der LiPo-Akkus erreicht.

Problem 3: Odometry für die Lokalisierung des Automobils:

Damit das Auto immer weiß, wo es sich momentan befindet, wird eine Technik namens Lokalisation angewendet. Diese sorgt für eine Beziehung zwischen Auto und Umgebung. Um diese Beziehung korrekt aufzustellen, braucht es das sogenannte ODOM. Dies ist ein Referenzframe, dass die Entfernung und Position des Autos im Verhältnis zu einem Startpunkt (z.B. Map-Frame) einschätzt. Normalerweise wird der ODOM von SLAM-Toolbox automatisch berechnet. Leider geht dies aufgrund von Problem 1 nicht bei uns. Wir müssen also unseren eigenen ODOM berechnen. Zum Glück gab es einige Ressourcen, die wir verwenden konnten. Hierbei ist aber ein großes Problem aufgetreten. Da die meisten Roboter einen anderen Mechanismus zur Lenkung des Roboters verwenden, gibt es nicht wirklich viele ODOMs für die sogenannte Ackermann-Lenkung. Wir haben eine Anleitung gefunden, die uns ein ODOM erzeugen lässt. Hierbei ist aber weit über einen Monat verloren gegangen. Der Grund dafür ist ein einzelnes „-„ Zeichen. Dies war benötigt, um eine Kalkulation für das ODOM korrekt auszuführen. Nach einem Monat suchen und probieren sind wir endlich auf diesen Kalkulationsfehler gestoßen und konnten so nun unser ODOM berechnen und das Auto lokalisieren.

Programmcode

Der Programmcode ist auf GitHub unter https://github.com/Jomo178/sara

öffentlich verfügbar.

Die Software ist in Python und C++ geschrieben und basiert auf dem Framework ROS2 (Robot Operating System 2). Sie steuert die autonomen Funktionen des RC-Autos, insbesondere:

- SLAM (Simultaneous Localization and Mapping) zur Kartenerstellung mit dem LiDAR
- Pathfinding und Navigation mit dem ROS2-Nav2-Stack
- Duplex-Kommunikation zwischen Raspberry Pi und Arduino zur Motor- und Sensorsteuerung
- Echtzeit-Auswertung der Ultraschallsensoren zur Hindernisvermeidung

Die einzelnen Module sind in ROS2-Nodes aufgeteilt und über Topics und Services miteinander verbunden. Die Software ist modular aufgebaut und lässt sich leicht erweitern oder anpassen.



Ultrasonic Ranging Module HC - SR04

Product features:

Ultrasonic ranging module HC - SR04 provides 2cm - 400cm non-contact measurement function, the ranging accuracy can reach to 3mm. The modules includes ultrasonic transmitters, receiver and control circuit. The basic principle of work:

(1) Using IO trigger for at least 10us high level signal,
(2) The Module automatically sends eight 40 kHz and detect whether there is a pulse signal back.
(3) IF the signal back, through high level , time of high output IO duration is the time from sending ultrasonic to returning.

Test distance = (high level time×velocity of sound (340M/S) / 2,

Wire connecting direct as following:

5V Supply

Trigger Pulse Input

Echo Pulse Output

0V Ground

Electric Parameter

Working Voltage	**DC 5 V**
Working Current	**15mA**
Working Frequency	**40Hz**
Max Range	**4m**
Min Range	**2cm**
MeasuringAngle	**15 degree**
Trigger Input Signal	**10uS TTL pulse**
Echo Output Signal	**Input TTL lever signal and the range in proportion**

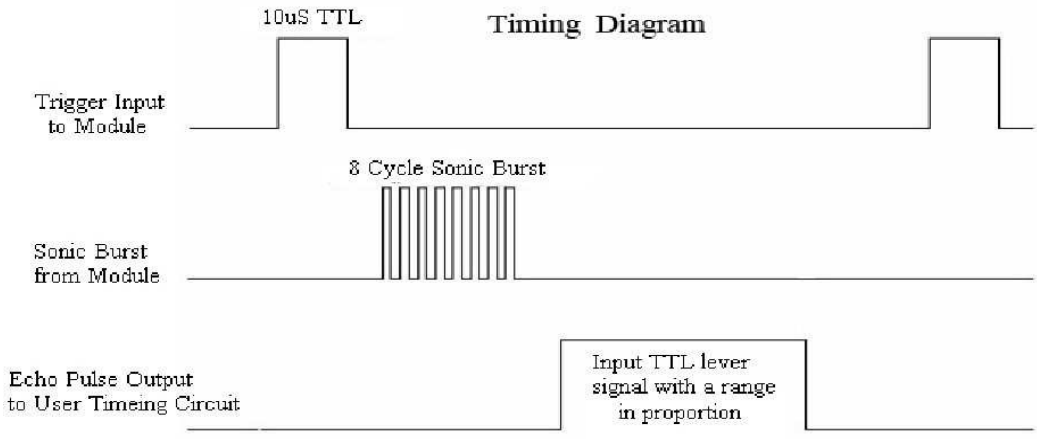
Dimension	**45*20*15mm**



Vcc Trig Echo GND

Timing diagram

The Timing diagram is shown below. You only need to supply a short 10uS pulse to the trigger input to start the ranging, and then the module will send out an 8 cycle burst of ultrasound at 40 kHz and raise its echo. The Echo is a distance object that is pulse width and the range in proportion. You can calculate the range through the time interval between sending trigger signal and receiving echo signal. Formula: uS / 58 = centimeters or uS / 148 =inch; or: the range = high level time * velocity (340M/S) / 2; we suggest to use over 60ms measurement cycle, in order to prevent trigger signal to the echo signal.



Attention:

The module is not suggested to connect directly to electric, if connected electric, the GND terminal should be connected the module first, otherwise, it will affect the normal work of the module.

When tested objects, the range of area is not less than 0.5 square meters and the plane requests as smooth as possible, otherwise, it will affect the results of measuring.

Quellen

Mega 2560 Rev3 | Arduino Documentation (Eingesehen am 20.06.25)

Datei:Ultraschallsensor HC SR04 Datenblatt.pdf – HSHL Mechatronik (Eingesehen am 20.06.25)

https://gpiozero.readthedocs.io/en/stable/installing.html (Eingesehen am 01.03.24)

https://www.doknow.de/iraspberry.html#:~:text=RPi.,einem%20IC-Pin%20angeschlossen%20ist (Eingesehen am 01.03.24)

https://wiki.youyeetoo.com/en/Lidar/LD20 (Eingesehen am 01.03.24)

https://github.com/gisbi-kim/PyICP-SLAM (Eingesehen am 05.03.24)

https://www.hackster.io/dmitrywat/slam-with-ros-using-bittle-and-raspberry-pi-4-dad850 (Eingesehen am 05.07.24)

https://tutorials-raspberrypi.de/arduino-raspberry-pi-miteinander-kommunizieren-lassen/ (Eingesehen am 09.10.24)

https://roboticsbackend.com/raspberry-pi-arduino-serial-communication/ (Eingesehen am 16.10.24)

https://www.youtube.com/watch?v=uuslVHPjWS8 (Eingesehen am 24.01.25)

https://stevengong.co/notes/F1TENTH (Eingesehen am 30.01.25)

https://github.com/CL2-UWaterloo/Raceline-Optimization (Eingesehen am 30.01.25)

https://github.com/CL2-UWaterloo/f1tenth_ws/tree/main/src/pure_pursuit (Eingesehen am 30.01.25)

https://answers.ros.org/question/398095/ (Eingesehen am 10.04.25)

https://robotics.stackexchange.com/questions/109615/drift-issue-while-doing-slam-using-slam-toolbox (Eingesehen am 10.04.25)

https://msadowski.github.io/hands-on-with-slam_toolbox/ (Eingesehen am 10.04.25)

https://discourse.articulatedrobotics.xyz/t/discussion-slam-on-our-robot-with-slam-toolbox-making-a-mobile-robot-pt-15/163/21?page=2 (Eingesehen am 10.04.25)

https://www.digikey.com/en/maker/tutorials/2021/how-to-control-servo-motors-with-a-raspberry-pi#:~:text=To%20make%20a%20Raspberry%20Pi,the%20power%20supply%20as%20well (Eingesehen am 14.09.25)

https://pypi.org/project/pynput/ (Eingesehen am 14.09.25)